Project 2 – Decision Trees, Random Forest, Linear Regression, Regression Trees

CS4445 Data Mining and Knowledge Discovery in Databases – D Term 2026

Prof. Carolina Ruiz

Students: Victor Che, Shane Garvin, Jacob Wiberg

Dataset Description, Exploration, and Initial Preprocessing: (at most 2 pages)

[05 points] Dataset Description: (e.g., dataset domain, number of instances, number of attributes, distribution of target attribute, attributes that have missing values, …)

The dataset is the Stroke Prediction Dataset from Kaggle, which is a healthcare / public health dataset intended to predict whether a patient has experienced a stroke from demographic, lifestyle, and medical variables. The file contains 5,110 instances and 12 attributes total, including the target variable stroke; equivalently, it has 11 input features plus 1 target. The attributes are: id, gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status, and stroke. The target attribute is binary, where stroke = 1 indicates stroke and stroke = 0 indicates no stroke. The class distribution is strongly imbalanced: about 249 stroke cases and 4,861 non-stroke cases, so only about 4.9% of instances belong to the positive class.

The dataset mixes numerical, binary, and categorical attributes. age, avg_glucose_level, and bmi are continuous numerical variables; hypertension, heart_disease, and stroke are binary; and variables such as gender, ever_married, work_type, Residence_type, and smoking_status are categorical. The main missing-data issue is in bmi, which has 201 missing values (about 3.9% of the dataset). In the Kaggle data description, smoking_status = "Unknown" is also noted to mean the information was unavailable, though that is a category value rather than a formal null. In our preprocessing code, we also remove the single gender = "Other" row so that the gender feature can be encoded as binary, and we drop the id field because it is only an identifier and not a predictive feature.

[10 points] Data Exploration: (e.g., comments on interesting or salient aspects of the dataset, visualizations, correlation, issues with the data, …)

One important aspect of the dataset is the strong class imbalance in the target variable. Only a small fraction of patients are labeled stroke = 1, so a naive classifier can achieve high accuracy by predicting the majority class almost all the time. This makes metrics such as precision, recall, AUC, and the confusion matrix more informative than accuracy alone. In the code, this is important enough that the preprocessing includes an option to upsample the minority class for classification.

Another salient feature is the presence of mixed attribute types. The dataset contains continuous medical variables such as age, avg_glucose_level, and bmi, binary indicators such as hypertension and heart_disease, and nominal categorical variables such as work_type and smoking_status. Because of this, preprocessing is necessary before modeling: identifiers are removed, categorical variables are converted with binary encoding or one-hot encoding, and feature names are standardized.

A notable data-quality issue is missing BMI values. In the classification pipeline, missing bmi values are filled with the median so that all instances can still be used. In the regression pipeline, rows with missing BMI are dropped because BMI itself is the prediction target and therefore cannot be imputed without leaking information. Another small issue is that the dataset contains a single gender = "Other" instance; in the code this row is removed so that gender can be encoded as a binary feature.

[05 points] Initial data preprocessing, if any, based on data exploration findings: (e.g., removing IDs, strings, …)
Specify whether the preprocessing was used for the classification task, the regression task or both.
Note: Keep initial data preprocessing to a minimum – just the necessary – and instead experiment with data preprocessing in the following sections.

For the regression task, the dataset was first copied and restricted to rows with non-missing bmi values, since BMI is the target variable being predicted and rows without a known target cannot be used for supervised regression. The single row with gender = "Other" was removed so that gender could be encoded consistently as a binary feature. The id attribute was dropped because it is only an identifier and has no predictive meaning. The feature Residence_type was renamed to residence_type for consistency, and binary categorical variables were encoded numerically: gender, ever_married, and residence_type were mapped to 0/1 values. The remaining nominal categorical variables, work_type and smoking_status, were one-hot encoded so they could be used by the machine learning models. After preprocessing, the target variable bmi was separated from the predictors, and the data was split into training and testing sets for downstream use. The code also includes an option to remove stroke from the regression features to avoid possible target leakage when predicting BMI.

For the classification task, preprocessing followed a similar structure but was adapted to the stroke prediction target. The data was copied, the single gender = "Other" row was removed, and the id column was dropped. Residence_type was again renamed to residence_type, and missing bmi values were filled with the median BMI so that observations would not be discarded unnecessarily. Binary categorical features (gender, ever_married, and residence_type) were converted to 0/1 form, while work_type and smoking_status were one-hot encoded. The target variable stroke was then separated from the predictors, and the data was split into training and testing sets. In addition, the classification preprocessing includes an optional upsampling step that resamples the minority stroke class within the training split to address the strong class imbalance in the dataset.

Code Description: Python Libraries and Functions you used and with what parameters. If you wrote your own Python code, describe it here. (at most 2 pages)

[10 points] Classification Techniques: (at most 1 page)

For classification preprocessing, the main libraries used were Pandas and scikit-learn. pandas was used for general data cleaning and feature transformation because it provides convenient DataFrame operations for dropping columns, renaming attributes, filling missing values, encoding categories, and combining features. scikit-learn was used for splitting the dataset and optionally handling class imbalance in the training data. The dataset itself was loaded through kagglehub, which provides direct access to the Kaggle dataset as a Pandas DataFrame.

The first important function was kagglehub.dataset_load(...), used with KaggleDatasetAdapter.PANDAS, the dataset name "fedesoriano/stroke-prediction-dataset", and the file path "healthcare-dataset-stroke-data.csv". This function loads the dataset directly from Kaggle into a Pandas DataFrame. Within preprocessing, Pandas DataFrame methods such as .drop(columns=['id']), .rename(columns={'Residence_type': 'residence_type'}), and .fillna(X['bmi'].median()) were used. .drop(...) removes irrelevant columns, here removing the identifier field id; .rename(...) changes a column name for consistency; and .fillna(...) replaces missing BMI values with the median BMI. The categorical binary features were encoded with .map({...}), which converts text categories into numeric form required by the models.

Another important preprocessing function was pd.get_dummies(...), used on the columns ['work_type', 'smoking_status'] with the parameters prefix=['work_type', 'smoking_status'] and dtype=int. pd.get_dummies(...) performs one-hot encoding, which turns each category into its own binary indicator column. After feature creation, the target variable was separated using y = X.pop('stroke'), which removes the stroke column from the predictor matrix and stores it as the classification target.

“Majority Class”:

sklearn.dummy.DummyClassifier with strategy='most_frequent' was used as a baseline estimate for evaluating our models. This method predicts the stroke based on the target value from all inputs, and ignores all additional features. (it is considered a very “dumb” predictor).

Decision Trees:

sklearn.tree.DecisionTreeClassifier was used with four parameter configurations: default, max_depth=5, criterion='entropy', splitter='random', and min_samples_split=10. The max_depth parameter limits tree complexity to reduce overfitting. The criterion parameter switches from the default Gini impurity to information gain (entropy).

Random Forests:

sklearn.ensemble.RandomForestClassifier was used with four configurations: default, max_depth=5, max_depth=50, and criterion='poisson'.

[10 points] Regression Techniques: (at most 1 page)

Preprocessing Techniques for Regression:

For regression preprocessing, the main libraries used were Pandas, scikit-learn, and kagglehub. kagglehub was used to load the dataset directly from Kaggle into a Pandas DataFrame. pandas was then used for most of the cleaning and feature-engineering work, and scikit-learn was used in this preprocessing stage only indirectly through the later modeling pipeline.

The dataset is first loaded using kagglehub.dataset_load(...) with KaggleDatasetAdapter.PANDAS, the dataset identifier "fedesoriano/stroke-prediction-dataset", and the file name "healthcare-dataset-stroke-data.csv". This function downloads and returns the Kaggle data as a Pandas DataFrame. Inside regression_pp(...), the first major operation is .dropna(subset=['bmi']), which removes all rows where BMI is missing. This is necessary because bmi is the target variable in the regression task, and supervised learning cannot train on observations with missing target values. The code then removes the single gender = "Other" row using boolean filtering, which allows the gender feature to be encoded consistently as binary. The identifier column is removed with .drop(columns=['id']), since it has no predictive value.

Additional feature preparation is done with several Pandas functions. .rename(columns={'Residence_type': 'residence_type'}) is used to standardize the feature name. The binary categorical variables are converted to numeric form using .map(...). The nominal categorical variables work_type and smoking_status are then encoded using pd.get_dummies(...) with the parameters prefix=['work_type', 'smoking_status'] and dtype=int. This function performs one-hot encoding.

Finally, the target variable is separated using y = X.pop('bmi'). The .pop(...) function removes the bmi column from the predictor DataFrame and returns it as a separate Series, which becomes the regression target. The preprocessing function also contains an optional argument include_stroke=True. If this is set to False, the code uses .drop(columns=['stroke']) to remove the stroke feature from the predictors, which can help avoid possible leakage or unwanted dependence if the experiment is intended to predict BMI without using stroke status.

Linear Regression:

sklearn.linear_model.LinearRegression was used to find a line of best fit for our dataset once the preprocessing discussed earlier was applied. We performed a variety of experiments that are described below (In later sections). We used this to help find the attributes that best predict strokes (positively or negatively) and find the strength of the correlations (weight of coefficients) for each when predicting our target (stroke or no stroke in patient).

Regression Trees:

sklearn.tree.DecisionTreeRegressor was used to create our regression trees. Each of which were created with a series of different parameters (also described below).

Guiding Questions

[10 points] Three Guiding Questions for the Classification Experiments: (at most 1/2 page)

1. **Which of these models would be the most efficient in predicting a stroke with the provided attributes?**
   This question aims to compare the performance of different classification models which are the Majority Class, Decision Trees, and Random Forest. After running each model using 10 fold cross validation, we found that the Majority class has the fastest and most efficient in predicting a stroke. Being the only double digit CV time (34.927 ms) while being the most accurate (0.95). However despite the high accuracy, it fails to detect positive due to class imbalance.

2. **Is the given dataset sufficient for predicting strokes, or is more data required in order to cover a meaningful amount of variance?**
   Based on the results from our experiments (described below) we were unfortunately not able to cover a majority of the variance without radically overfitting the data, which would have prevented our predictive model from generalizing. Although we would not claim to have performed our data mining operations perfectly, this would suggest that a larger dataset would have produced better results.

3. **Which model variation would give the most accurate response of each model?**
   Each model variation has a different variant (e.g. Decision tree [default, max_depth=5, etc] / Random Forest [default, entropy, etc]. However, the majority of the model variations are not correct due to an imbalance. The most accurate response for random forest is max_depth=10 and Decision Tree is max_depth=5

[10 points] Three Guiding Questions for the Regression Experiments: (at most 1/2 page)

1. **Can the available attributes in this data set meaningfully predict BMI with more accuracy than our baseline estimate?**
   Somewhat. RandomForestRegressor(max_depth=5) achieved the best results overall with $R^2 = 0.281$ and RMSE = 6.66. That said, even the best model leaves roughly 72% of BMI variance unexplained. This would suggest that we need more information in order to make a more accurate estimate going forward.

2. **If we can accurately make a prediction, which attributes are the best predictors? What does this tell us in terms of general recommendations for patients who seek to make healthy choices?**

   The features most likely to carry predictive weight for BMI are age, avg_glucose_level, hypertension, and heart_disease. This unfortunately does not give much practical information on how to directly adjust your BMI. However, It does give some insight into how general heart health (diet and exercise) impact your well being, in the sense that many of these negative consequences come bundled together. That is to say it is rare (from a data standpoint) to maintain a high BMI and have no impact on your glucose and hypertension values.

3. **Which training factors / parameters best optimize our predictions.**

   The single most impactful parameter was max_depth in the Random Forest Regressor. Constraining depth to 5 improved R^2 from 0.198 (unconstrained) to 0.281 and cut training time from ~9,700 ms to ~2,800 ms — a

simultaneous gain in both accuracy and efficiency. This would unfortunately lead us to believe we have an Over Fitting problem and would likely benefit from a very large increase in data entries and possible attributes.

[50 points] Summary of Classification Experiments Use 10-fold cross-validation. *At most 1.5 pages total. Run **at least** 1 experiment with the Majority Class Classifier, 5 different experiments with Decision Trees, and 5 different experiments with Random Forests.*

For each experiment describe: (1) ML technique used (e.g., Decision Trees or Random Forest); (2) Classification guiding questions used (e.g., "1", "2", and/or "3"); (3) any pre-processing used; (4) Parameters; (5) Evaluation results: Accuracy, Precision, Recall, ROC Area, Time to build model, Size of model; (6) Analysis & observations about experiment, and interesting patterns in the trees or models (readability).

For these experiments, the general preprocessing explained above was used, then upsampling was applied during 10-fold cross-validation. This was an attempt to minimize the challenge posed by a large class imbalance.

Model	Variant	CV Time (ms)	Nodes	Depth	Leaves	Accuracy	Precision	Recall	AUC
Majority Class	—	34.927	—	—	—	0.9513	0.0000	0.0000	0.5000
Decision Tree	Default	241.887	729	17	365	0.9152	0.1290	0.1285	0.5420
Decision Tree	max_depth=5	175.315	61	5	31	0.7461	0.1268	0.7149	0.7720
Decision Tree	entropy	236.297	677	20	339	0.9109	0.1098	0.1165	0.5341
Decision Tree	random splitter	133.686	1233	23	617	0.9115	0.1020	0.1044	0.5286
Decision Tree	min_samples_split=10	221.126	469	16	235	0.9172	0.1405	0.1365	0.5469

Random Forest	Default	5556.690	—	—	—	0.9411	0.0938	0.0241	0.7927
Random Forest	entropy	6278.430	—	—	—	0.9419	0.1364	0.0361	0.8007
Random Forest	n_estimators=200	11606.31	—	—	—	0.9419	0.1250	0.0321	0.8072
Random Forest	n_estimators=50	4292.396	—	—	—	0.9403	0.1000	0.0281	0.7845
Random Forest	max_depth=10	6080.554	—	—	—	0.8812	0.1734	0.3815	0.8139

[20 points] Summary of Python Classification Results, Analysis, Discussion, and Visualizations (at most 1.5 pages)
(1) Analyze the effect of varying parameters/experimental settings on the results. (2) Analyze the results from the point of view of the dataset domain, and discuss the answers that the experiments provided to your guiding questions. (3) Include (a part of) the best classification model obtained.

The results suggest that predictive signal does exist but is relatively weak and unevenly captured by the models. While most models achieve high overall accuracy (~0.90–0.95), this is misleading due to the strong class imbalance. More informative metrics such as AUC show only moderate separability, with the best models (random forests and constrained decision trees) reaching around 0.80–0.81. This indicates the features contain some predictive structure, but not enough for highly reliable classification of the minority class.

The results demonstrate the challenges posed by class imbalance. The majority class classifier achieves the highest accuracy but completely fails to identify any positive cases, as reflected by zero precision and recall. Decision trees show a tradeoff between depth and minority class detection: deeper trees maintain higher accuracy but have poor recall, while the shallow tree (max_depth=5) significantly improves recall (0.7149) and AUC, indicating much better detection of positive cases at the cost of overall accuracy.

Random forests seem to achieve the highest AUC values, but most configurations still struggle with very low recall. The max_depth=10 random forest stands out as a more balanced model, achieving a substantial increase in recall (0.3815) along with the highest AUC (0.8139), though with reduced accuracy. Overall, limiting model complexity appears to improve the ability to capture the minority class. For decision trees, limiting depth (e.g., max_depth=5) dramatically improves AUC and recall compared to fully grown trees, suggesting better generalization and improved minority-class

detection. For random forests, the max_depth=10 configuration performs best overall in terms of balancing AUC and recall, showing that regularization of tree growth is more impactful than simply increasing ensemble size or using different splitting criteria. Overall, the best-performing configurations are those that prevent overly complex trees from overfitting to the majority class.

[50 points] Summary of Regression Experiments. Use 10-fold cross-validation. *At most 1.5 pages.*

*Run **at least** 1 experiment with Majority Class Classifier, 2 different experiments with Linear Regression, 4 different experiments with Regression Trees, and 3 different experiments with Random Forests.*

For each experiment describe: (1) ML technique used (e.g., Linear regression, Random Forests); (2) Regression guiding questions used (e.g., "1", "2", and/or "3"); (3) any pre-processing used; (4) Parameters; (5) Evaluation results: Correlation Coefficient and other Error Metric(s) (specify what metrics you used), Time to build model, Size of model; (6) Analysis & observations about experiment, and interesting patterns in the trees or models (readability).

Model	Variant	CV Time (ms)	Corr. Coef.	MSE	RMSE	MAE	R^2
Majority Class	—	7.385	-0.0675	61.7438	7.8577	5.9793	-0.0011
Linear Regression	Default	40.316	0.4834	47.2689	6.8752	5.1097	0.2336
Linear Regression	fit_intercept=False	46.056	0.4834	47.2689	6.8752	5.1097	0.2336
Decision Tree	Default	174.322	0.3010	90.0362	9.4887	7.0850	-0.4598
Decision Tree	poisson	220.646	0.2922	91.7096	9.5765	7.1226	-0.4869

Decision Tree	random splitter	100.332	0.2852	90.7324	9.5254	7.0984	-0.4711
Random Forest	Default	9282.228	0.4712	49.4734	7.0337	5.2693	0.1979
Random Forest	max_depth=5	2814.177	0.5297	44.3728	6.6613	4.9344	0.2806
Random Forest	n_estimators=50	4710.193	0.4631	50.2299	7.0873	5.3034	0.1856
Random Forest	poisson	11707.958	0.4717	49.4147	7.0296	5.2558	0.1988

[20 points] Summary of Python Regression Results, Analysis, Discussion, and Visualizations (at most 1.5 pages) 1. Analyze the effect of varying parameters/experimental settings on the results. 2. Analyze the results from the point of view of the dataset domain, and discuss the answers that the experiments provided to your guiding questions. 3. Include (a part of) the best regression model obtained.

The most impactful parameter was max_depth in the Random Forest Regressors. Constraining to max_depth=5 improved R^2 from 0.198 to 0.280 and reduced training time from 9691 ms to 2950 ms, making it the best regression model overall. For single Decision Tree Regressors, all configurations (default, Poisson criterion, random splitter) severely overfit, producing R^2 values near -0.48 to -0.52. The criterion choice (squared error vs. Poisson) had negligible impact in both trees and forests, answering guiding question 3 negatively: Poisson criterion does not help for this problem. Reducing n_estimators from 100 to 50 had only a small negative effect on performance.

Domain analysis: The moderate R^2 values across all models (best: 0.28) indicate that BMI is inherently difficult to predict from demographic and lifestyle data alone. Features like age, glucose level, smoking status, work type, and marital status capture some sociological and health signals correlated with BMI, but the majority of BMI variance (>70%) is unexplained by these variables. This aligns with domain knowledge: BMI is influenced heavily by genetics, diet, and exercise habits that are not captured in this dataset. Linear regression performs comparably to or better than unconstrained tree-based models, suggesting the underlying relationship between these features and BMI is approximately linear at this level of feature granularity. Depth-constrained Random Forests improve on linear regression modestly (R^2 0.28 vs 0.23) by capturing some non-linear interactions.

Best model: The RandomForestRegressor(max_depth=5) is the recommended regression model. It achieves the best R^2 (0.2800), best RMSE (6.664 BMI units), and best MAE (4.931 BMI units) of all configurations tested, while training approximately 3.3x faster than the unconstrained Random Forest. The depth constraint prevents individual trees from overfitting noisy training instances, and the ensemble average reduces variance significantly. For interpretability, LinearRegression() is a close second (R^2 = 0.234) with near-instant training time and fully explainable coefficients.

Authorship: Although each student on the team is expected to be involved in every aspect of the project, describe in detail here the main contributions that each of the team members made to this project. This authorship description must accurately reflect the work done by each team member, and must be approved by all of the members of the team (at most 1/3 page)

Jacob:

- **Took the lead on implementing preprocessing for regression and classification, and creating evaluation functions for both strategies.**
- **Edited classification and regression for consistency**
- **Merged work for uniformity**

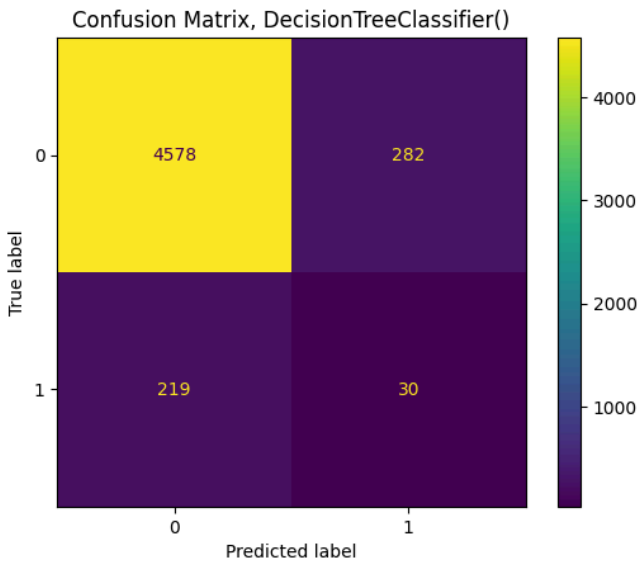
Shane:

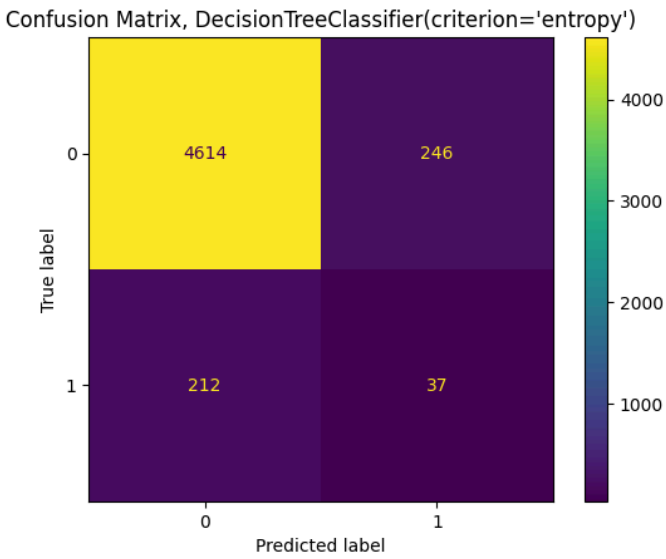
- **Organized communication and helped devise plan of action and divide responsibilities (along with Jacob)**
- **Planned and implemented regression portion of of code**
- **Composed majority Regression and classification segments of this report**

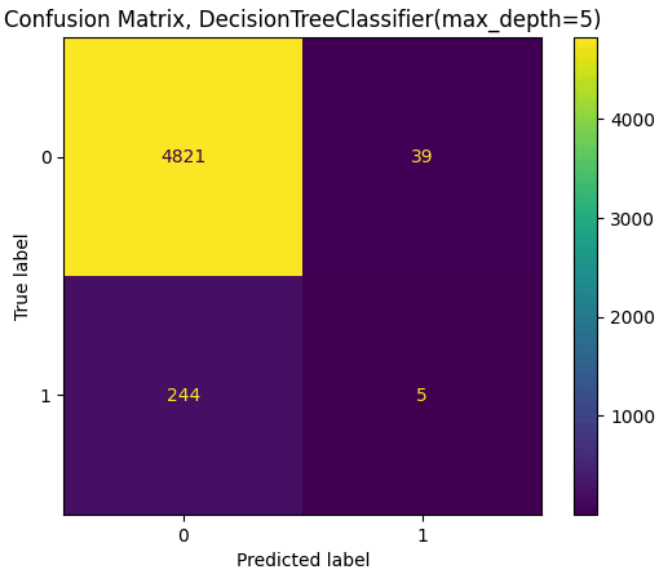
Victor:

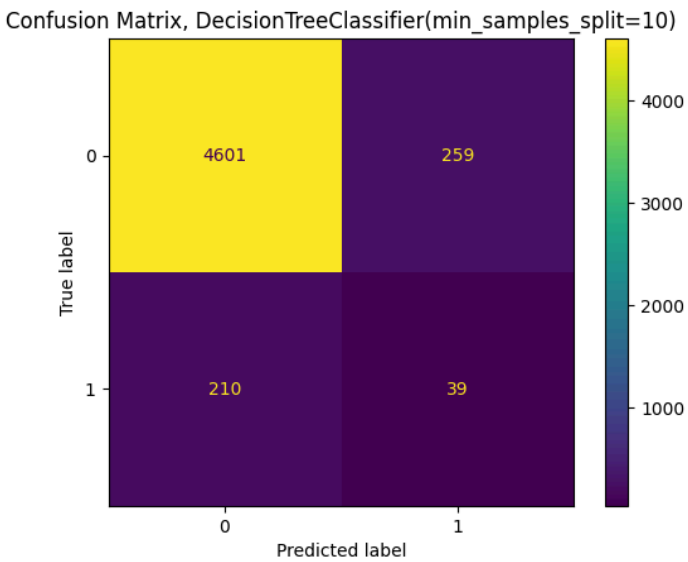
- **Developed classification segment of python script**

Appendix

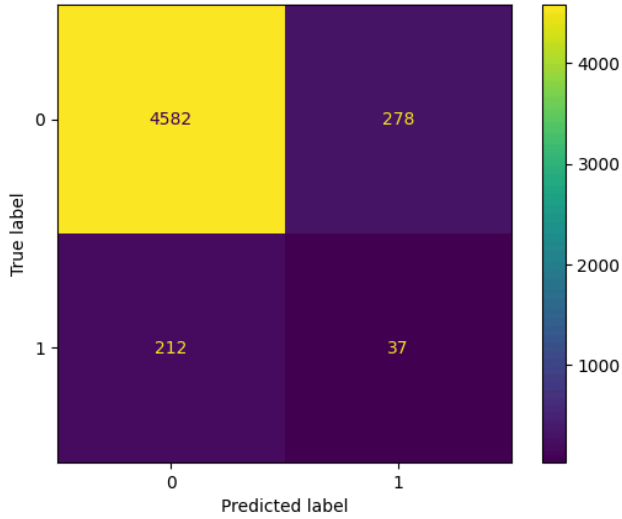

Confusion Matrix, DecisionTreeClassifier()
4578
282
219
30
True label
Predicted label
0
1
4000
3000
2000
1000


Confusion Matrix, DecisionTreeClassifier(criterion='entropy')
4614
246
212
37
True label
Predicted label
0
1
4000
3000
2000
1000

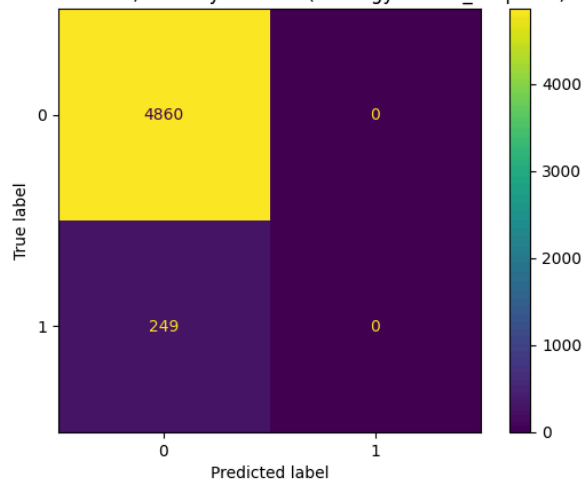

Confusion Matrix, DecisionTreeClassifier(max_depth=5)
4821
39
244
5
True label
Predicted label
0
1
4000
3000
2000
1000


Confusion Matrix, DecisionTreeClassifier(min_samples_split=10)
4601
259
210
39
True label
Predicted label
0
1
4000
3000
2000
1000

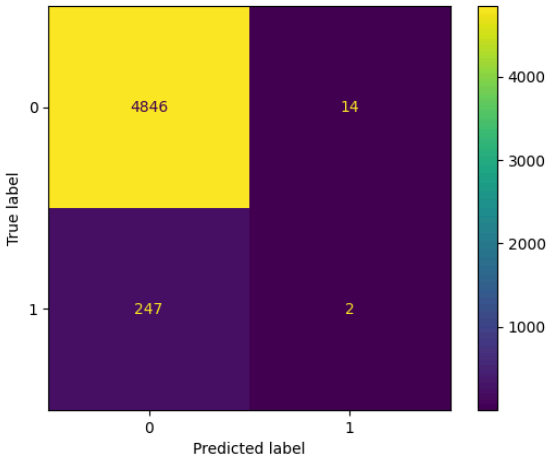
Confusion Matrix, DecisionTreeClassifier(splitter='random')



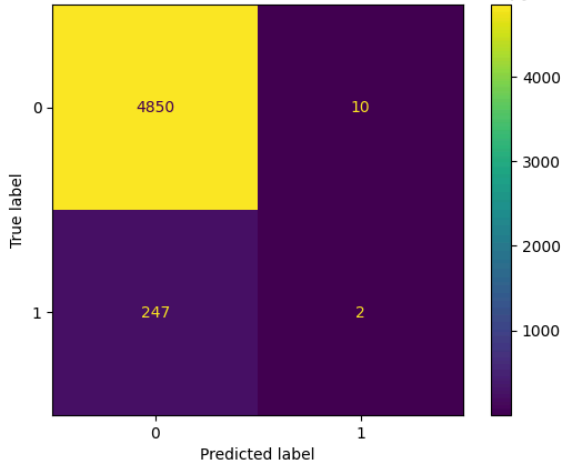
Confusion Matrix, DummyClassifier(strategy='most_frequent')



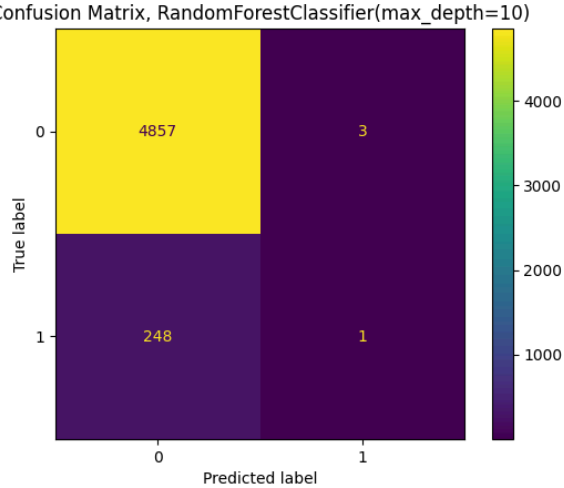
Confusion Matrix, RandomForestClassifier()



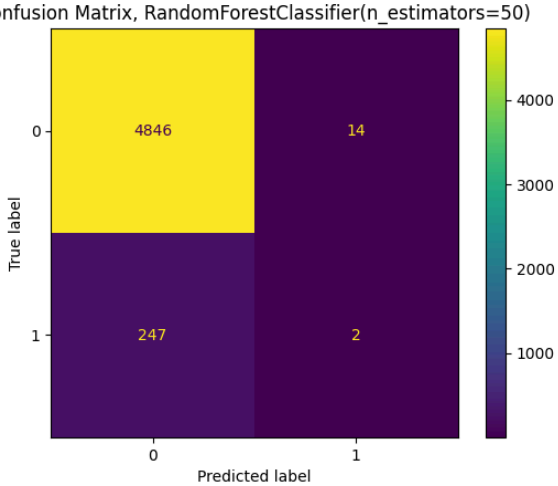
Confusion Matrix, RandomForestClassifier(criterion='entropy')

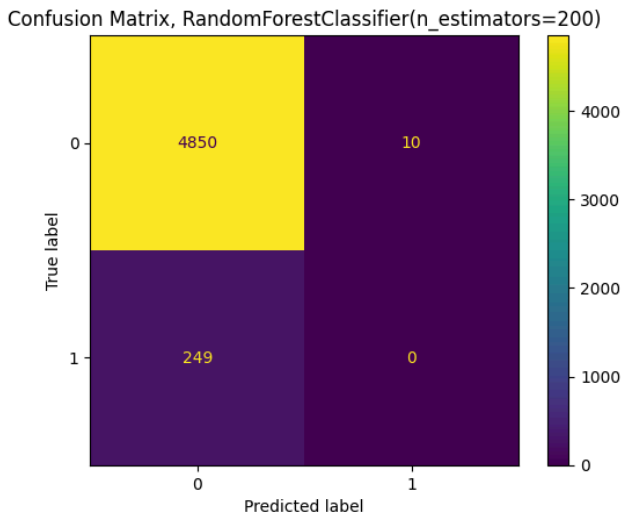


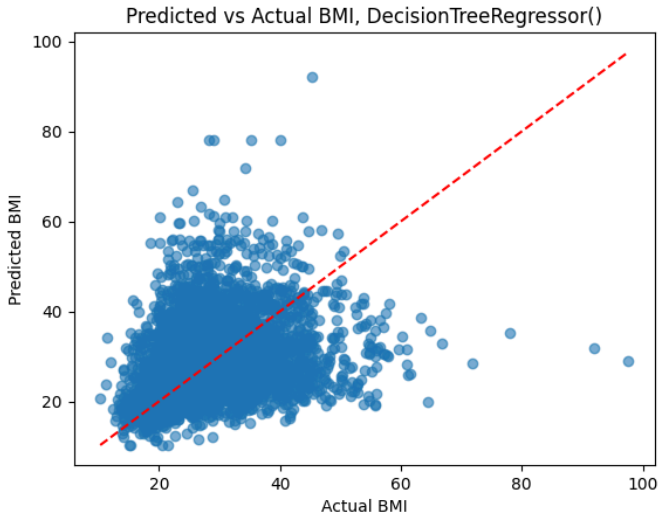
Confusion Matrix, RandomForestClassifier(max_depth=10)

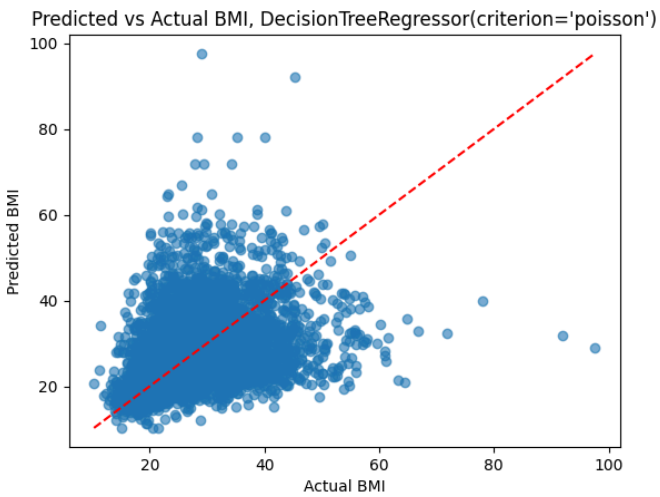


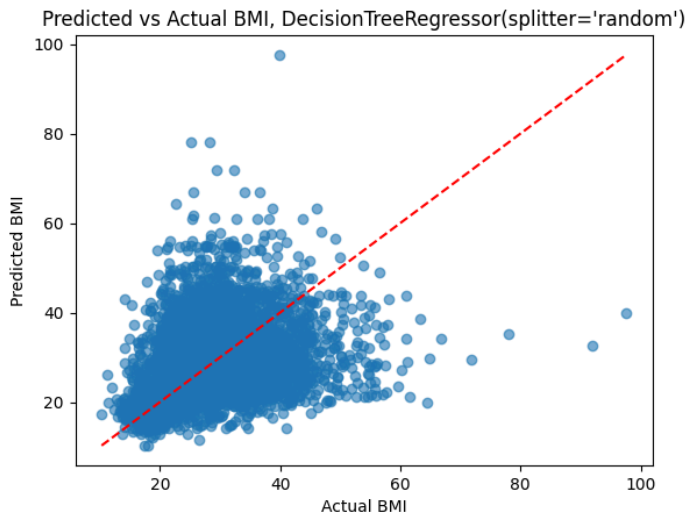
Confusion Matrix, RandomForestClassifier(n_estimators=50)

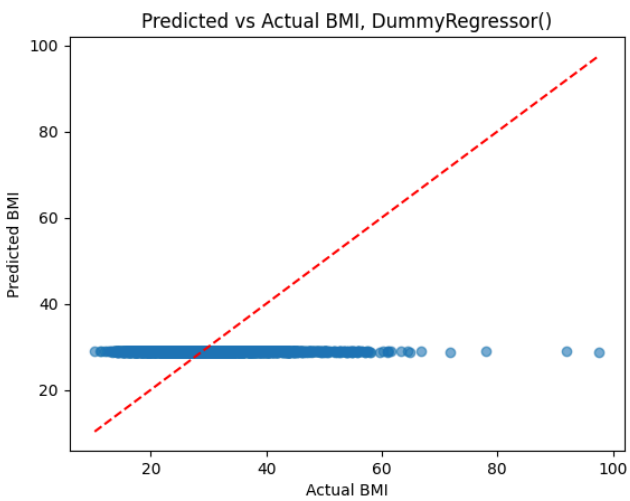


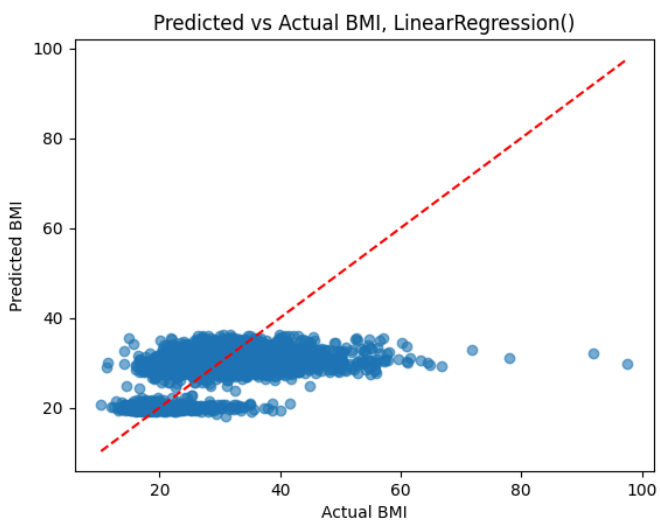

Confusion Matrix, RandomForestClassifier(n_estimators=200)
4850
10
249
0
True label
Predicted label
0
1
4000
3000
2000
1000

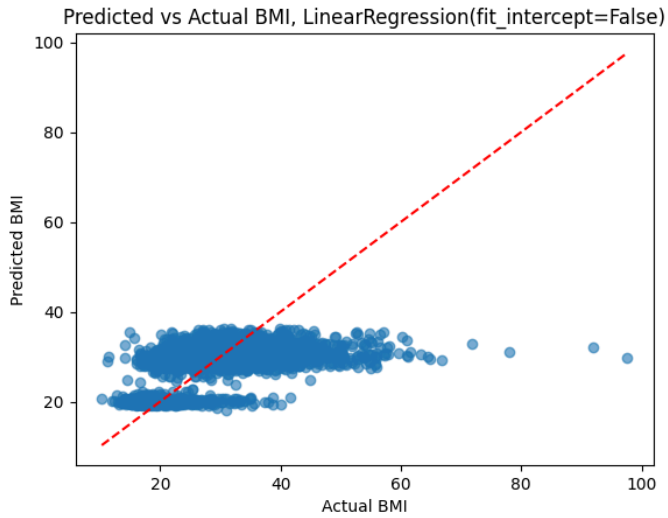

Predicted vs Actual BMI, DecisionTreeRegressor()
Predicted BMI
Actual BMI

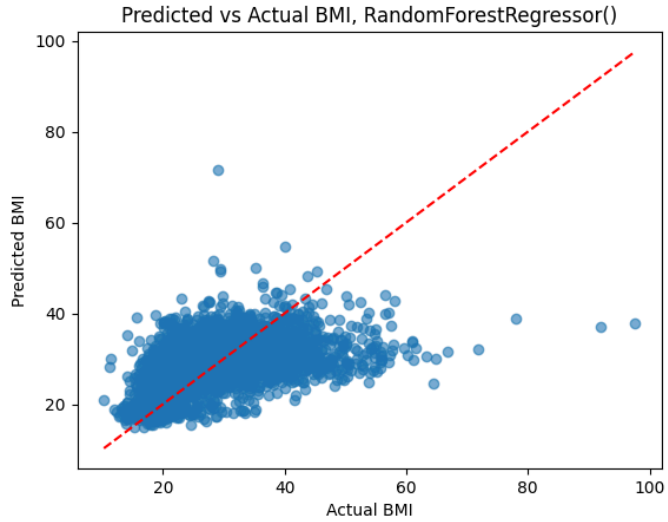

Predicted vs Actual BMI, DecisionTreeRegressor(criterion='poisson')
Predicted BMI
Actual BMI

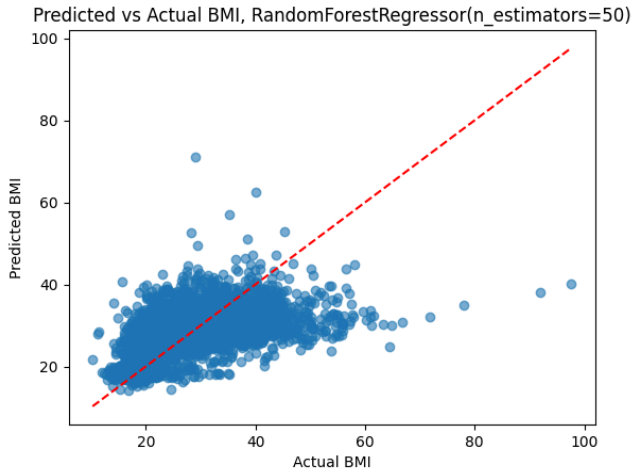

Predicted vs Actual BMI, DecisionTreeRegressor(splitter='random')
Predicted BMI
Actual BMI

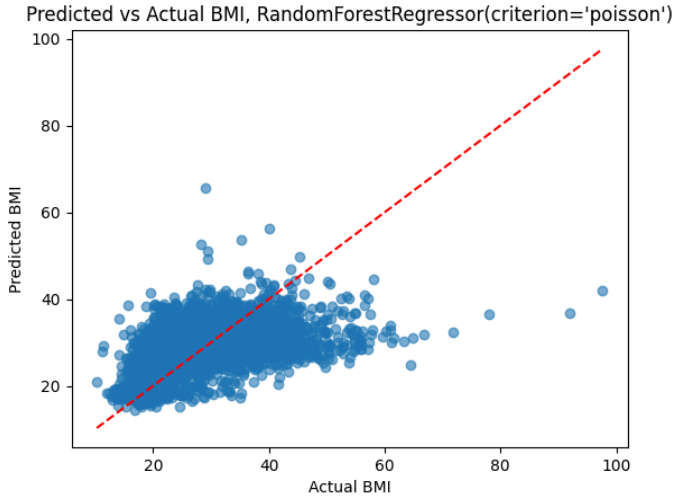

Predicted vs Actual BMI, DummyRegressor()
Predicted BMI
Actual BMI

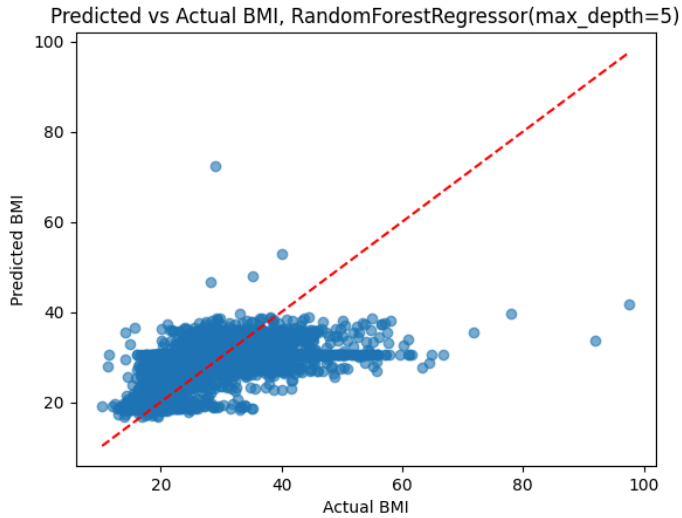

Predicted vs Actual BMI, LinearRegression()
Predicted BMI
Actual BMI


Predicted vs Actual BMI, LinearRegression(fit_intercept=False)
Predicted BMI
Actual BMI


Predicted vs Actual BMI, RandomForestRegressor()
Predicted BMI
Actual BMI


Predicted vs Actual BMI, RandomForestRegressor(n_estimators=50)
Predicted BMI
Actual BMI


Predicted vs Actual BMI, RandomForestRegressor(criterion='poisson')
Predicted BMI
Actual BMI


Predicted vs Actual BMI, RandomForestRegressor(max_depth=5)
Predicted BMI
Actual BMI